

Part 2 – Technical Explanation

GymFlow: Architecture, Trade-offs & Design Decisions

Architecture Overview

GymFlow is a standalone Angular 17 application — built with Angular's standalone components rather than NgModules — backed by Firebase: Firestore for data, Firebase Authentication for staff login, and Firebase Hosting for deployment. The project is organised by responsibility rather than by feature module: **pages/** holds the dashboard and login views, **components/** holds reusable pieces such as the member form, **services/** holds **auth.service.ts** and **member.service.ts** for Firebase interaction, **models/** holds TypeScript interfaces, and **guards/** protects routes from unauthenticated access. This keeps Firestore logic out of components and behind a service layer, so the UI never talks to Firestore directly.

Firestore holds two collections: **members** (name, contact, planId, startAt, expiresAt) and **plans** (name, months, price), referenced by ID rather than embedded, so plan pricing stays editable in one place. Firebase Authentication gates all reads and writes through Firestore Security Rules, so member data is never publicly exposed.

Scaling to 10,000 Users a Day

As built, the app loads the members list in a single query, which works at demo scale but would not hold up at 10,000 users. Moving to production, I would add pagination using Firestore cursors instead of loading the full collection, add a composite index on **expiresAt** for the expiry-filter query before it becomes a bottleneck, and cache plan lookups in memory since plans change rarely but are read constantly. None of this is implemented yet — it is the next step, not a current feature.

Trade-offs

Decision	Reason	Trade-off
Firebase over a custom REST API	Faster to build; real-time out of the box	Vendor lock-in; harder to migrate later
Plain CSS (no UI library)	Full control over branding	More CSS to write upfront
No automated tests	Scope constraint within the time-box	Reduced confidence in edge-case behaviour
Email/Password auth only	Simplest to implement and demo	No SSO / Google login for staff convenience

Deliberately Left Out

Given the time-box, the following were out of scope but would be prioritised next: payment tracking per member, SMS/email notifications for expiring memberships, role-based access control (admin vs. front-desk staff), an audit log of who modified which record and when, and bulk import of members via CSV. Of these, I would start with expiry notifications first, since flagging expiring memberships on a dashboard only helps if someone is actively looking at it.